TRNG Module (`trng.v`) – Reference Documentation

Project: PicoRV32 SoC on Real Digital Boolean board (Spartan-7 XC7S50) **Modules:** `trng` (bus slave) and `trng_ro_bank` (ring-oscillator entropy source) | **Base address:** `0x5000_0000` | **Bus:** PicoRV32 native memory interface

---

1. Overview

`trng` is a memory-mapped **True Random Number Generator**. Its entropy comes from the timing jitter of 16 free-running ring oscillators built from FPGA look-up tables. The raw bits pass through start-up discard, continuous health tests and optional von Neumann de-biasing, and are then assembled into 32-bit words that software reads from a single `DATA` register.

Key design points:

- **Physical entropy source** (ring oscillators), not a pseudo-random generator.
- **Health monitoring** with two NIST SP 800-90B style tests; output is blocked automatically if the source fails.
- **Safe software model:** reading `DATA` pops the word; an empty or failed read returns 0 and sets a sticky UNDERFLOW flag, so stale or bad data is never returned silently.
- **Optional interrupt** when a word is ready or a health test fails.

> **Honest scope statement.** The defaults (`DIV` , `RCT_CUTOFF` , `APT_CUTOFF`) are conservative starting points, **not a certified entropy claim**. Real entropy depends on the board and placement. Validate with captured raw data (section 8) before using the output for security-critical purposes.

Quick facts

Item	Value
Base address	`0x5000_0000` (top.v decodes `mem_addr[31:28] == 4'h5`)

Item	Value
Registers	DATA (0x0), STATUS (0x4), CTRL (0x8), DIV (0xC)
Entropy source	16 ring oscillators, lengths 3, 5, 7, … 33 inverting stages (288 LUTs total)
Raw sample rate	`f_clk / (DIV + 1)`; default DIV = 255 gives 390,625 raw bits/s at 100 MHz
Output	32-bit words, first generated bit lands in bit 31
Reset polarity	**Active LOW** in the current source (see section 11)
Power-on state	Disabled (`EN = 0`), von Neumann on, interrupt off, DIV = 255
Interrupt	Level: `IRQ_EN & (DATA_VALID
Bus latency	1 wait state, registered read data

---

2. Architecture

The data path is a straight pipeline. Everything after the synchronisers runs in the system clock domain.

Stage	Block	What it does
1	Ring-oscillator bank	16 independent oscillators, enabled by `EN`
2	Synchronisers	Two flip-flops per oscillator (`ASYNC_REG`), resolve metastability
3	XOR	All 16 synchronised outputs XORed into one bit
4	Sampler	One sample every `DIV + 1` clock cycles
5	Warm-up	First `WARMUP_SAMPLES` (1024) samples are discarded
6	Health tests	Repetition Count Test and Adaptive Proportion Test on the raw bits
7	Conditioner	Optional von Neumann de-biasing

Stage	Block	What it does
8	Word assembler	Shift register builds a 32-bit word
9	Output register	One-word holding register, read through `DATA`

The health tests (stage 6) monitor the **raw** bits before conditioning, so they judge the entropy source itself rather than the cleaned output. A one-cycle delay between stages 6 and 7 guarantees that a failing sample sets the fail flag *before* that sample can reach the output.

---

3. Entropy source (`trng_ro_bank`)

3.1 Ring oscillators

Oscillator `i` (i = 0 to 15) is a loop of `3 + 2i` inverting LUT stages, so lengths are 3, 5, 7, ..., 33 (always odd, which is required for oscillation).

- Stage 0 is a `LUT2` configured as **NAND(feedback, EN)**. With `EN = 1` it acts as an inverter and the loop oscillates; with `EN = 0` its output is forced high and the loop stops. This is how the oscillators are switched on and off.
- Stages 1 to LEN-1 are `LUT1` inverters.
- Different lengths give different frequencies, so the oscillators do not lock to one another.
- The total is 16 NAND stages plus 272 inverter stages = **288 LUTs**.
- All loop nets and LUTs carry `KEEP`, `DONT_TOUCH` and `ALLOW_COMBINATORIAL_LOOPS` attributes so synthesis does not optimise the loops away.

The randomness comes from accumulated jitter: tiny, unpredictable variations in each oscillator's edge timing. Sampling an oscillator with an unrelated clock captures its phase at that instant; with a long enough sampling interval and many oscillators XORed together, the result is unpredictable.

3.2 Simulation model

Zero-delay combinational loops cannot oscillate in an RTL simulator. The loops are therefore compiled **only when the `SYNTHESIS` macro is defined** (Vivado defines it). In simulation each oscillator output is replaced by a random bit updated every clock using `$random`. Consequences:

- Simulation verifies the **logic** (registers, tests, interrupts), not the entropy quality.
- Simulation output is pseudo-random and must never be used to judge the real randomness.

---

4. Sampling, rate and latency

4.1 Sample period

The sampler takes one bit every `DIV + 1` clock cycles. A larger `DIV` lets more jitter accumulate between samples, which improves the entropy per bit but lowers throughput. `DIV` can be changed at any time and takes effect immediately.

4.2 Expected throughput (100 MHz clock, unbiased source)

With von Neumann on, only pairs of different bits produce an output bit, so the expected output is about one quarter of the raw rate. A biased source yields less.

DIV	Sample period	Raw bits/s	Output bits/s (VN on)	Words/s (VN on)	Words/s (VN off)
63	64 cycles	1,562,500	about 390,600	about 12,200	about 48,800
255 (default)	256 cycles	390,625	about 97,700	about 3,050	about 12,200
1023	1024 cycles	97,656	about 24,400	about 760	about 3,050
4095	4096 cycles	24,414	about 6,100	about 190	about 760

4.3 Start-up latency (default settings)

After `EN` is set, the first 1024 samples are discarded: 1024 x 256 cycles = 262,144 cycles, about **2.6 ms** at 100 MHz. Collecting the first 32 output bits then takes about 128 samples with von Neumann on (about 0.33 ms). The first word is therefore available roughly **3 ms** after enabling. Later words follow at the rates in the table above.

4.4 Behaviour while a word is waiting

The holding register stores one word. While it is full (`DATA_VALID = 1`), new samples are **dropped** (not queued) but health testing continues. After you read the word, assembly of the next word starts from fresh samples, so a delay in reading never causes stale bits to be reused.

---

5. Health tests

Both tests run on the raw sampled bits once warm-up has finished, and only while `EN = 1`. Either failure sets a **sticky** flag.

5.1 Repetition Count Test (RCT)

Counts consecutive identical bits. If a run reaches `RCT_CUTOFF` (default **32**), `RCT_FAIL` is set. This catches a stuck-at source (all zeros or all ones). Under the NIST formula `C = 1 + ceil(20 / H)`, a cutoff of 32 corresponds to an assumed min-entropy of about 0.65 bit per sample. A genuinely random 50/50 source would need a run of 32 to occur by chance, which is vanishingly unlikely.

5.2 Adaptive Proportion Test (APT)

Uses a window of `APT_WINDOW` (default **1024**) samples. The first sample in the window is the reference; the test counts how many samples in the window equal it (the reference counts as 1). If the count reaches `APT_CUTOFF` (default **680**), `APT_FAIL` is set. For an ideal unbiased source the expected count is about 512 with a standard deviation near 16, so 680 is over ten standard deviations away and false alarms are negligible. In practice the test flags a source whose bit probability has drifted to roughly 65 to 70 percent or more.

5.3 What happens on failure

While either flag is set:

- the word being assembled and any waiting word are **discarded**, and assembly is held in reset;
- reads of `DATA` return 0 and set `UNDERFLOW`;
- the interrupt is raised if `IRQ_EN = 1`;
- the flags stay set until software writes `CLR` (or the SoC is reset).

A `CLR` restarts the warm-up and the tests. If the source is still bad, the test will fail again.

5.4 Tuning the cutoffs

The defaults are conservative. After measuring the raw min-entropy of your board (section 8), recompute the RCT and APT cutoffs from NIST SP 800-90B for that entropy value and set the `RCT_CUTOFF` and `APT_CUTOFF` parameters accordingly.

---

6. Conditioning and word format

6.1 Von Neumann de-biasing (`VN_EN = 1`, default)

Samples are taken in non-overlapping pairs:

Pair (first, second)	Output
0, 1	bit 0
1, 0	bit 1
0, 0 or 1, 1	discarded

This removes a constant bias if consecutive samples are independent. It cannot repair correlation between samples, and it costs throughput (about one quarter of the raw rate when unbiased).

6.2 Raw mode (`VN_EN = 0`)

Every sampled bit goes straight into the word. Use this for characterising the source (section 8), not for production randomness.

6.3 Word format

Bits are shifted in MSB-first: the **first** bit generated ends up in bit 31 and the **last** in bit 0. After a successful read the output register is cleared to 0.

---

7. Registers

All registers are 32 bits, word aligned.

Address	Name	Access	Reset value	Summary
`0x5000_0000`	`TRNG_DATA`	RO, **read pops**	`0x0000_0000`	Random word
`0x5000_0004`	`TRNG_STATUS`	RO	`0x0000_0000`	Status and sticky flags
`0x5000_0008`	`TRNG_CTRL`	RW	`0x0000_0002`	Enable, mode, interrupt, clear

Address	Name	Access	Reset value	Summary
`0x5000_000C`	`TRNG_DIV`	RW	`0x0000_00FF`	Sample period setting

7.1 `TRNG_DATA` – offset 0x0

Bits	Description
31:0	Random word

- If a word is available **and** no health test has failed: returns the word and clears `DATA_VALID` (the read pops it).
- Otherwise: returns `0x0000_0000` and sets `UNDERFLOW`.
- **A read has a side effect.** Do not read `DATA` speculatively or from a debugger unless you intend to consume the word.
- Writes are ignored.

7.2 `TRNG_STATUS` – offset 0x4 (read-only, no side effects)

Bit	Name	Meaning
0	`DATA_VALID`	A fresh word is waiting in `DATA`
1	`HEALTH_FAIL`	`RCT_FAIL` or `APT_FAIL` is set (sticky)
2	`RCT_FAIL`	Repetition Count Test failed (sticky)
3	`APT_FAIL`	Adaptive Proportion Test failed (sticky)
4	`WARMUP_DONE`	Start-up discard has finished
5	`UNDERFLOW`	`DATA` was read while empty or failed (sticky)
31:6	reserved	Read as 0

Typical values: `0x10` = running, warm-up done, no word yet; `0x11` = word ready; `0x1E` = both tests failed.

7.3 `TRNG_CTRL` – offset 0x8

Bit	Name	Access	Reset	Description
0	`EN`	RW	0	1 = oscillators and sampling run. 0 = everything stops, test state and data are flushed, warm-up restarts on next enable
1	`VN_EN`	RW	1	1 = von Neumann de-biasing; 0 = raw bits
2	`IRQ_EN`	RW	0	Enables the interrupt output
3	`CLR`	W (self-clearing)	0	Write 1: clears fail flags and `UNDERFLOW`, flushes data, restarts warm-up. Always reads 0
31:4	reserved			Ignored on write, 0 on read

> **Warning – a CTRL write replaces EN, VN_EN and IRQ_EN with the bits you write** (only byte 0 is used). Writing `0x08` to clear failures also sets `EN = 0`, `VN_EN = 0` and `IRQ_EN = 0`. To clear while keeping the normal configuration, write the full value with bit 3 added, for example `0x0B` (`EN | VN_EN | CLR`).

7.4 `TRNG_DIV` – offset 0xC

Bits	Field	Description
15:0	`DIV`	Sample period = `DIV + 1` clock cycles. Reset: 255
31:16	reserved	Read as 0

Byte-wise writes are supported. `DIV = 0` samples every clock and gives the oscillators almost no time to accumulate jitter; it is not recommended.

7.5 Address decoding

The module decodes `addr[4:2]` only, so the four registers **repeat every 32 bytes** across the `0x5xxx_xxxx` region (for example `0x5000_0020` is another alias of `DATA`, and reading it also pops a word). Offsets `0x10` to `0x1C` read as 0. Always use the four documented addresses.

---

8. Software guide

8.1 C definitions

```
#include <stdint.h>

#define TRNG_BASE    0x50000000u
#define TRNG_DATA    (*(volatile uint32_t *)(TRNG_BASE + 0x0))
#define TRNG_STATUS  (*(volatile uint32_t *)(TRNG_BASE + 0x4))
#define TRNG_CTRL    (*(volatile uint32_t *)(TRNG_BASE + 0x8))
#define TRNG_DIV     (*(volatile uint32_t *)(TRNG_BASE + 0xC))

#define TRNG_EN        (1u << 0)
#define TRNG_VN_EN     (1u << 1)
#define TRNG_IRQ_EN    (1u << 2)
#define TRNG_CLR       (1u << 3)

#define ST_DATA_VALID  (1u << 0)
#define ST_HEALTH_FAIL (1u << 1)
#define ST_RCT_FAIL    (1u << 2)
#define ST_APT_FAIL    (1u << 3)
#define ST_WARMUP_DONE (1u << 4)
#define ST_UNDERFLOW   (1u << 5)
```

8.2 Initialise and read a word (polling)

```
void trng_init(void) {
    TRNG_DIV  = 255;                          /* default; raise for more
entropy per bit */
    TRNG_CTRL = TRNG_EN | TRNG_VN_EN;         /* start, de-biasing on, no
interrupt     */
}

/* returns 0 on success, -1 if the health tests have failed */
int trng_read(uint32_t *out) {
    for (;;) {
        uint32_t st = TRNG_STATUS;
        if (st & ST_HEALTH_FAIL) return -1;
        if (st & ST_DATA_VALID) {
            *out = TRNG_DATA;                 /* pops the word */
            return 0;
        }
    }
}
```

8.3 Recovering from a health failure

```
TRNG_CTRL = TRNG_EN | TRNG_VN_EN | TRNG_CLR;   /* keep config, clear flags,
restart warm-up */
/* wait for ST_WARMUP_DONE (about 2.6 ms at defaults), then read again */
```

If failures repeat, the oscillators are not providing enough entropy: increase `DIV`, check the constraints in section 9, and re-characterise.

8.4 Interrupt use

Set `TRNG_IRQ_EN` in the CTRL value. In the SoC the line is connected to `irq[4]` of PicoRV32; unmask it with the `maskirq` custom instruction, with the vector at `0x1000_0010`. The line is a level: it stays high while a word is waiting or a failure is flagged, and drops when the word is read (or after `CLR` for failures). The handler should read `STATUS` first to tell the two causes apart.

8.5 Characterising the source (recommended before trusting it)

1. Write `TRNG_CTRL = TRNG_EN` (VN off, raw mode) and read several hundred thousand words (about 1 million bits or more) through the UART.
2. Run the NIST SP 800-90B `ea_non_iid` min-entropy estimator on the raw bits; also run `ent` or dieharder.
3. Repeat with von Neumann on and compare.
4. Raise `DIV` until the estimated min-entropy per raw bit is comfortably above your assumed value, then recompute `RCT_CUTOFF` and `APT_CUTOFF`.
5. As a quick sanity check that the data is not deterministic, reset the board several times: the first words after each reset must differ.

---

9. Vivado and synthesis notes

These are required for the ring oscillators to survive implementation. They belong in your constraints file and are **not** included in the RTL:

```
# combinational loops are intentional
set_property SEVERITY {Warning} [get_drc_checks LUTLP-1]
# the oscillator outputs are asynchronous to clk: do not time them
set_false_path -through [get_nets -hier -filter {NAME =~
*u_trng/u_bank/g_ro*/n[*]}]
```

The net-name pattern may need adjusting to your hierarchy. Further notes:

- Oscillator frequency depends on placement and routing, so results can differ between builds and between boards. Re-validate after any significant change.
- The synchroniser registers carry `ASYNC_REG`, which keeps each pair together and treats the crossing correctly.
- Because the oscillators stop when `EN = 0`, leaving the TRNG disabled when unused saves power and switching noise.

---

10. SoC integration (`top.v`)

```
trng u_trng (
    .clk(clk), .reset(resetn),                  // active-LOW reset: connect
resetn directly
    .valid(mem_valid & trng_sel), .ready(trng_ready),
    .wstrb(mem_wstrb), .addr(mem_addr), .wdata(mem_wdata),
    .rdata(trng_rdata), .irq(trng_irq)
);
```

- `trng_sel = (mem_addr[31:28] == 4'h5)`.
- `mem_ready` is the OR of the core, timer and TRNG ready signals; `mem_rdata` selects `trng_rdata` when `trng_sel` is true.
- `trng_irq` feeds `irq[4]`.
- Read data is **registered** at the access, so the pop side effect and the data returned always agree.

Parameters

Parameter	Default	Meaning
`NUM_RO`	16	Number of ring oscillators (lengths 3 + 2i)
`WARMUP_SAMPLES`	1024	Samples discarded after enable or clear
`RCT_CUTOFF`	32	Consecutive identical bits that fail the RCT
`APT_WINDOW`	1024	Samples per APT window
`APT_CUTOFF`	680	Reference-bit count that fails the APT
`DIV_RESET`	255	Reset value of the `DIV` register

---

11. Verification status, known issues and notes

Verified in simulation (Icarus Verilog, exact `trng.v` provided, with active-low reset): random-looking words with about 50 percent ones over 200 words; STATUS values; underflow flag; interrupt; forced stuck source triggering `RCT_FAIL` and `APT_FAIL`; data blocked during failure; recovery with `CLR`; `DIV` read and write. The simulation uses the behavioural oscillator model, so it proves the logic only.

Hardware observation: a captured log of 82 consecutive words from the board showed balanced bits (50.8 percent ones), no repeated words, full GF(2) rank and uniform bytes, with no sequential pattern. That is encouraging but far too small to certify quality (section 8.5).

Known issues and cautions

1. **Reset polarity and stale comments.** The code uses `if (!reset)` throughout, so the module is in reset when `reset = 0`. Connect it to `resetn`. The header and port comment in the file still say "active HIGH"; update them to avoid confusion.
2. **CTRL write replaces EN, VN_EN, IRQ_EN.** See the warning in section 7.3.
3. **Reading `DATA` consumes the word,** including through aliased addresses (`0x5000_0020` and so on).
4. **No FIFO.** Only one word is buffered; bits generated while it is full are dropped.
5. **Entropy is not certified.** Defaults are conservative guesses; characterise and tune.
6. **Placement dependence.** The oscillators are not hard-placed, so quality can vary between builds.
7. **No cryptographic post-processing.** Output is conditioned only by optional von Neumann. For keys, feed the words into a standard cryptographic DRBG or hash conditioner in software rather than using them directly.